Tarea Unidad 4: Programación de Scripts de Administración y Automatización (Curso 25/26)

1. DESCRIPCIÓN GENERAL. OBJETIVOS

Esta tarea consolida las competencias de programación dentro del servidor de bases de datos (procedimientos, funciones y disparadores) y su interacción con los metadatos del SGBD para labores de administración.

Para la realización de esta entrega, utilizaremos la base de datos de la **"Liga de Baloncesto"** (liga), cuya estructura de tablas (equipos, jugadores, partidos, estadisticas) debió ser desplegada previamente.

2. Bloque I: Estructuras Básicas, Funciones y Control de Flujo (Repaso)

Ejercicio 1: Procedimiento Sumador

- **Enunciado:** Diseña un procedimiento almacenado sp_sumar_valores al cual le pases dos valores numéricos enteros como parámetros de entrada (IN) y devuelva la suma de ambos en una variable de salida (OUT).

Ejercicio 2: Función Sumadora

- **Enunciado:** Crea una función almacenada fn_sumar_valores que tenga exactamente la misma funcionalidad del ejercicio anterior (recibir dos enteros y retornar su suma).

Ejercicio 3: Control de Signo y Paridad

- **Enunciado:** Crea un procedimiento sp_analizar_suma que invoque internamente al procedimiento o función de los ejercicios anteriores. Debe recibir dos números de entrada y devolver dos parámetros de salida:

- El primer parámetro de salida (OUT) devolverá un carácter representativo: '>' si la suma es positiva, '<' si es negativa, o '=' si es igual a cero.
- El segundo parámetro de salida (OUT) será un valor booleano (TINYINT(1) o BOOLEAN) que indique si la suma es par (TRUE/1) o impar (FALSE/0).

Ejercicio 4: Contador de Homónimos

- **Enunciado:** Escribe una función fn_contar_jugadores_nombre que reciba como parámetro el nombre de un jugador y devuelva el número de jugadores que se llaman exactamente así dentro de la tabla jugadores.

Ejercicio 5: Bucle de Capitanes

- **Enunciado:** Crea un procedimiento sp_contar_por_capitan que emplee una estructura de bucle repetitivo (WHILE, LOOP o REPEAT) para calcular y mostrar por pantalla cuántos jugadores tienen asignado como capitán de equipo a los IDs del 1 al 14 de forma secuencial.

3. Bloque II: Estructuras Avanzadas - Cursores

Ejercicio 6: Cursor de Resultados Límites (Puntuación de 90)

- **Enunciado:** Diseña un procedimiento sp_partidos_puntuacion_limite que utilice un **CURSOR** para recorrer los partidos. El procedimiento debe imprimir o visualizar el nombre de los equipos enfrentados y sus respectivos resultados únicamente cuando alguno de los dos equipos haya alcanzado exactamente los 90 puntos en el encuentro.

Ejercicio 7: Búsqueda del Máximo Salario sin funciones de agregado

- **Enunciado:** Crea un procedimiento sp_jugador_max_salario que utilice un **CURSOR** para obtener los nombres de los jugadores y sus salarios. El procedimiento debe calcular y mostrar el nombre del jugador con el salario más alto del club **sin utilizar la función predefinida MAX()**.

Ejercicio 8: Equipo con Mayor Actividad

- **Enunciado:** Crea un procedimiento sp_equipo_mas_partidos que utilice un **CURSOR** para analizar el número de partidos jugados por cada equipo. Debe mostrar el nombre del

equipo que más encuentros ha disputado a lo largo de la liga **sin utilizar la función MAX()**.

Ejercicio 9: Procedimiento con Cursor para Modificación de Salarios (Mantenimiento)

- **Enunciado:** Crea un procedimiento sp_incrementar_salario_jugadores con un cursor al que se le pasará un porcentaje o una cantidad fija. El cursor deberá recorrer a todos los jugadores de la base de datos e ir incrementando su salario de forma individual.

4. Bloque III: Automatización mediante Disparadores (Triggers)

Ejercicio 10: Control de Salarios Negativos

- ```
  Enunciado: Crea un disparador
  tg_salario_positivo_before_update que controle la actualización de
  salarios sobre la tabla jugadores. En caso de que se intente asignar un
  salario menor o igual a 0, el disparador debe cancelar el cambio
  manteniendo el salario anterior del jugador de manera silenciosa, o
  lanzar un error personalizado.
  ```

Ejercicio 11: Actualización Dinámica de Clasificación

- ```
  Enunciado: Crea un disparador
  tg_actualizar_puntos_equipos de tipo AFTER INSERT sobre la tabla
  partidos. Cada vez que se inserte un partido, el disparador debe
  identificar cuál ha sido el equipo vencedor y sumarle de forma
  automática 1 punto en su casillero de clasificación en la tabla equipos.
  Realiza pruebas de inserción para verificar su funcionamiento.
  ```

Ejercicio 12: Auditoría Integral de la Tabla Equipos

- **Enunciado:** Diseña un sistema de auditoría para monitorizar los cambios en la tabla equipos.

1. ```
   Crea una tabla de logs llamada auditoria_equipos con los campos:
   id_log (PK, Auto), usuario, operacion (INSERT / UPDATE / DELETE),
   fecha_hora e id_equipo_afectado.
   ```

2. ```
   Crea tres disparadores (AFTER INSERT, AFTER UPDATE y AFTER DELETE)
   que alimenten de forma automática esta tabla registrando quién realizó
   la operación, la fecha exacta y la tabla afectada.
   ```

Ejercicio 13: Restricción de Inserciones por Privilegio de Usuario

- **Enunciado:** Crea un disparador tg_insert_solo_root para la tabla equipos. El disparador debe validar si el usuario que está realizando la operación es el administrador supremo (root@localhost). Si cualquier otro usuario intenta realizar un insert, el trigger debe abortar la operación utilizando SIGNAL SQLSTATE '45000' con un mensaje descriptivo de denegación de privilegios.

5. Bloque IV: Administración e Interacción con el Catálogo

Ejercicio 14: Replicador del Comando Describe (DESC)

- **Enunciado:** Diseña un procedimiento almacenado llamado sp_describir_tabla que reciba como parámetro de entrada el nombre de una tabla. El procedimiento debe consultar las vistas del sistema en information_schema.columns para mostrar por pantalla: el nombre de la columna, el tipo de dato, si permite nulos (YES/NO) y su valor predeterminado (reproduciendo de forma personalizada la salida de la orden DESCRIBE o DESC).

Ejercicio 15: Purga Programada de Partidos Antiguos

- **Enunciado:** Desarrolla un procedimiento sp_purgar_partidos al cual se le pasará una fecha como parámetro de entrada. El procedimiento eliminará de forma segura todos los registros de partidos disputados con anterioridad a la fecha indicada para reducir el peso en tablas históricas.